

01 / IOT & ROBOTICS

ESP32 Web Controlled Car Robot

| Wireless SoftAP Control & Embedded Asynchronous Web Dashboard.

ACTUATOR 26

FIELD NOTES / 2026

02 / SYSTEM ARCHITECTURE

4 Core Layers of the Web Car

From browser touch interactions down to motor hardware drivers:

WIFI ACCESS POINT**ESP32 SoftAP Mode**

Creates standalone Wi-Fi network (SSID: ROBOCEK-CAR) allowing direct phone/PC connections without a router.

HTTP SERVER**Asynchronous WebServer**

Hosts HTML/CSS/JS dashboard on Port 80, serving touch controls and processing /move & /speed endpoints.

WEB DASHBOARD**Touch & WASD Interface**

Embedded HTML UI with multi-touch buttons, WASD keyboard bindings, and interactive speed sliders.

ACTUATION**TB6612FNG Driver**

Dual H-Bridge MOSFET driver converting software move directives into differential motor PWM outputs.

03 / SCHEMATIC DIAGRAM

Circuit Diagram & Hardware Interconnects

SCHEMATIC PLACEHOLDER

ESP32 Dev Board + TB6612FNG Motor Driver

Insert final schematic / Fritzing wiring diagram here. Showing Wi-Fi antenna, motor control pins, and LiPo battery connections.

ESP32 SoftAP
ROBOCEK-CAR

Web Server
Port 80 (HTTP)

TB6612FNG Driver
STBY: 23 | PWM: 33/25

Default Speed
200 PWM



HTTP Endpoints & REST Route Architecture

The ESP32 WebServer exposes three light HTTP REST endpoints.

ENDPOINT ROUTE	HTTP METHOD	QUERY PARAMETERS
/	GET	None
/move	GET	?dir=forward backward left right stop
/speed	GET	?value=0..255

Motor Driver

... Wi-Fi antenna, ...
... ns.

Default Speed
200 PWM



05 / PIN MAPPING

ESP32 Pinout Assignment

Web Car Config

Exact GPIO assignments configured in web_controlled_car/src/main.cpp:

TB6612FNG DRIVER PINS

RC_MOTOR_STBY	GPIO 23 (Standby Enable)
RC_LEFT_PWM	GPIO 33 (Left Speed PWM)
RC_LEFT_IN1 / IN2	GPIO 18 / GPIO 19
RC_RIGHT_PWM	GPIO 25 (Right Speed PWM)
RC_RIGHT_IN1 / IN2	GPIO 26 / GPIO 27

WIFI & SERVER CONFIG

WIFI SSID	ROBOCEK-CAR
PASSWORD	12345678
IP ADDRESS	192.168.4.1 (Default AP)
DEFAULT SPEED	200 (PWM 0-255)

06 / CODE BREAKDOWN

Directional Motor Driving Functions

Core motor driving helpers from web_controlled_car/src/main.cpp:

```
// Left & Right Motor Controls with Speed Signed Polarity
```

```
void leftMotor(int speed) {  
    if (speed > 0)          { digitalWrite(RC_LEFT_IN1, HIGH); digitalWrite(RC_LEFT_IN2, LOW); }  
    else if (speed < 0)     { digitalWrite(RC_LEFT_IN1, LOW);  digitalWrite(RC_LEFT_IN2, HIGH); }  
    else                    { digitalWrite(RC_LEFT_IN1, LOW);  digitalWrite(RC_LEFT_IN2, LOW); }  
    analogWrite(RC_LEFT_PWM, abs(speed));  
}
```

```
void forward() { leftMotor(motorSpeed); rightMotor(motorSpeed); }  
void backward() { leftMotor(-motorSpeed); rightMotor(-motorSpeed); }  
void left()     { leftMotor(-motorSpeed); rightMotor(motorSpeed); }  
void right()    { leftMotor(motorSpeed); rightMotor(-motorSpeed); }  
void stopCar()  { leftMotor(0); rightMotor(0); }
```

07 / CODE BREAKDOWN

Asynchronous HTTP Request Handlers

Parsing incoming URL query arguments in `handleMove()` and `handleSpeed()`:

```
// HTTP Endpoint Handlers

void handleMove() {
  String direction = server.arg("dir");
  if (direction == "forward")    forward();
  else if (direction == "backward") backward();
  else if (direction == "left")  left();
  else if (direction == "right") right();
  else                          stopCar();

  server.send(200, "text/plain", "OK");
}

void handleSpeed() {
  if (server.hasArg("value")) {
    motorSpeed = constrain(server.arg("value").toInt(), 0, 255);
  }
  server.send(200, "text/plain", "OK");
}
```

08 / CODE BREAKDOWN

Wi-Fi SoftAP Setup & Server Launch

Creating standalone Access Point & binding HTTP routes in setup():

```
// setup() from web_controlled_car/src/main.cpp

void setup() {
  Serial.begin(115200);
  // Pin Modes & STBY HIGH...
  digitalWrite(RC_MOTOR_STBY, HIGH);
  stopCar();

  WiFi.mode(WIFI_AP);
  WiFi.softAP(SSID, PASSWORD);
  Serial.print("AP IP: "); Serial.println(WiFi.softAPIP());

  server.on("/", handleRoot);
  server.on("/move", handleMove);
  server.on("/speed", handleSpeed);
  server.begin();
}
```

09 / CODE BREAKDOWN

Zero-Delay Execution

server.handleClient()

Main loop execution remains 100% non-blocking:

```
// Non-Blocking Loop Execution  
  
void loop() {  
    server.handleClient(); // Process incoming HTTP client requests immediately  
}
```

GOLDEN RULE**Never Call delay() in a WebServer Loop!**

Calling delay() inside loop() causes Wi-Fi packet drops and severe control latency on mobile touch dashboards.

10 / WEB UI DASHBOARD

Responsive UI & Touch Optimization

TOUCH EVENTS

ontouchstart / ontouchend

Uses touch event listeners instead of standard `click` to trigger instant motor responses on mobile devices.

ASYNC FETCH

Asynchronous `fetch()` API

Issues background HTTP requests (`fetch("/move?dir=forward")`) without reloading the HTML page.

KEYBOARD BINDINGS

WASD & Arrow Keys

JavaScript `keydown` and `keyup` listeners enable desktop keyboard drive controls.

SCROLL LOCK

touchmove Prevention

Disables default touch-scrolling (`e.preventDefault()`) so dragging on screen won't pull the browser window.

11 / TROUBLESHOOTING

4 Web Car Field Diagnostics

ISSUE 1**Wi-Fi Disconnects**

ESP32 SoftAP antenna is sensitive to battery voltage sags. Ensure clean 5V/3.3V logic supply to ESP32 board.

ISSUE 2**Captive Portal Popup**

Some smartphones expect internet access on Wi-Fi. Disable "Switch to Mobile Data" setting on phone.

ISSUE 3**Motor Command Latency**

If car continues driving after releasing button, check that `ontouchend="stop()"` is registered on touch elements.

ISSUE 4**Stall at Low Speed**

If motors hum at low speed slider settings, increase minimum slider threshold to 60 PWM.



12 / READY TO CONTROL

Connect. Drive. Control.

Your ESP32 Web-Controlled Car is ready for wireless mobile dashboard control.

WEB CONTROL · ESP32 · ACTUATOR 26

